

SOFTWARE ENGINEERING LAB – 3

Component Modelling & Architectural Pattern Selection

Project: Incident Escalation & On-Call Rotation

Samrudh C Patil
PES1UG24CS417

1. Problem Statement / Lab 1 Context

The system is designed for incident escalation and on-call rotation. It receives an incident alert, identifies the active on-call engineer from the rotation schedule, sends the alert to that engineer, and records the incident status and outcome. If the on-call engineer does not acknowledge the incident within 5 minutes, the system triggers escalation and routes the incident to the secondary/on-call engineer. The escalation event is also recorded for tracking.

2. Architectural Pattern Selection

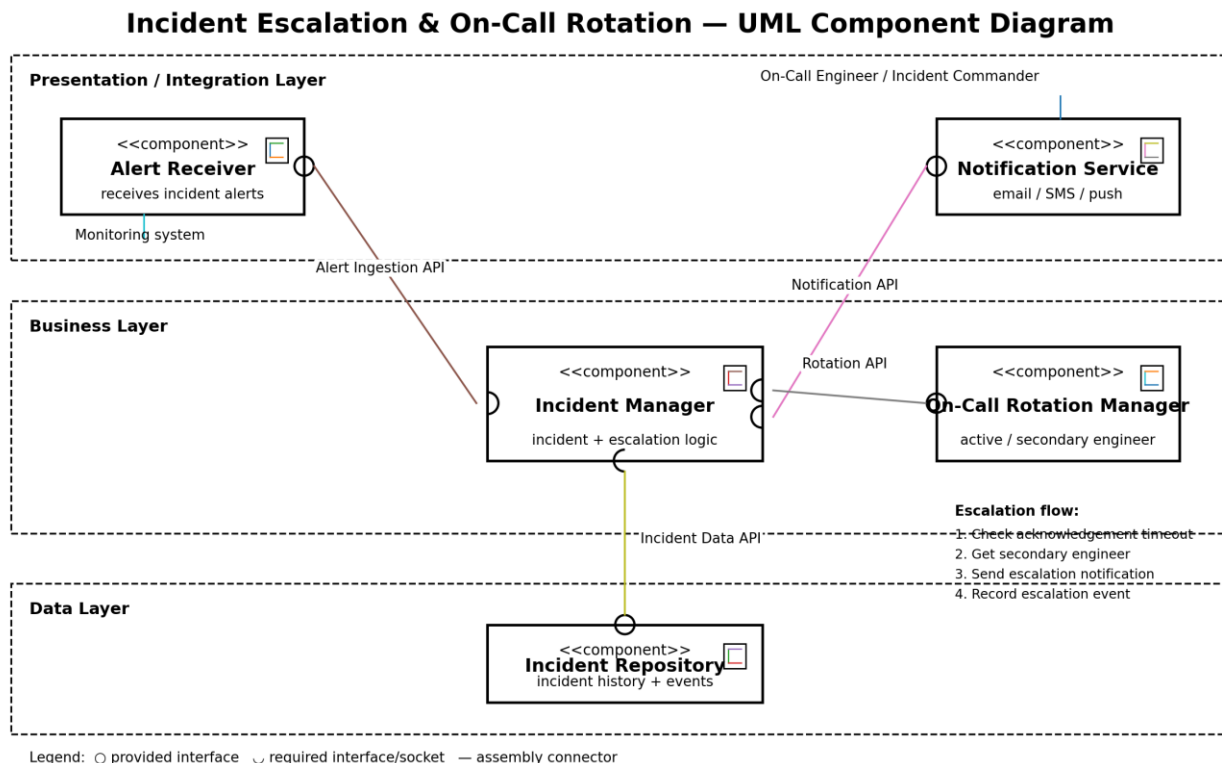
Selected Architecture: Layered Architecture

The system is organized into Presentation/Integration, Business, and Data layers. The components in each layer have focused responsibilities and communicate through defined interfaces.

3. Component Identification

Layer	Component	Responsibility
Presentation / Integration	Alert Receiver	Receives incident alerts from the monitoring system.
Presentation / Integration	Notification Service	Sends incident and escalation notifications to engineers.
Business	Incident Manager	Manages incidents, acknowledgement, status and escalation logic.
Business	On-Call Rotation Manager	Identifies the active and secondary/on-call engineers.
Data	Incident Repository	Stores incident history, status and escalation events.

4. UML Component Diagram



5. Interfaces and Component Interactions

Interface	Connection	Purpose
Alert Ingestion API	Alert Receiver → Incident Manager	Transfers incoming incident alerts for processing.
Notification API	Incident Manager → Notification Service	Sends incident alerts and escalation notifications.
Rotation API	Incident Manager → On-Call Rotation Manager	Retrieves the active or secondary engineer from the rotation.
Incident Data API	Incident Manager → Incident Repository	Stores and retrieves incident status, history and escalation events.

6. Architectural Justification

Architecture Selection

We chose Layered Architecture for the Incident Escalation & On-Call Rotation System.

Architectural Choice

The system is organized into Presentation/Integration, Business, and Data layers. The component diagram separates alert intake and notifications from incident-management logic and persistent incident data. This matches the Lab 1 workflow for receiving alerts, identifying the active on-call engineer, notifying and acknowledging incidents, escalating unacknowledged incidents, and recording incident history.

Two Scenario-Related Reasons

1. Separation of concerns: Alert reception, notification, incident management, on-call rotation, and data storage have separate responsibilities. This makes the system easier to understand and maintain.
2. Maintainability and controlled dependencies: The business layer communicates with the data layer through the Incident Data API. Changes to the storage implementation can therefore be made without changing the main incident-management workflow.

Security Advantage

Layer separation allows access to incident data to be controlled through the business layer. External alert sources and users do not need direct access to the Incident Repository. Validation and authorization can be handled at the appropriate service boundaries.

Performance Benefit

The architecture separates incident-processing operations into focused components. Alert processing, on-call lookup, notification, and data storage can be optimized independently. This avoids unnecessary direct interaction between external clients and the data layer.

7. Escalation Flow

1. The system receives an incident alert.
2. The Alert Receiver passes the alert to the Incident Manager.
3. The Incident Manager identifies the active on-call engineer through the On-Call Rotation Manager.
4. The Notification Service sends the incident alert to the active engineer.
5. The engineer acknowledges and handles the incident within 5 minutes.
6. If the incident is not acknowledged within 5 minutes, the Incident Manager triggers escalation.
7. The On-Call Rotation Manager provides the secondary/on-call engineer.
8. The Notification Service sends the escalation notification.
9. The Incident Repository records the incident status, outcome and escalation event.

8. Conclusion

Layered Architecture provides a clear separation between integration, business logic and data storage for the Incident Escalation & On-Call Rotation System. The component model defines the major modules and their interfaces while supporting the required incident acknowledgement, five-minute escalation, notification and incident-history workflows.